

---

# Debugging

# Debugging != Testing

---

Source code debugging is often undertaken in an IDE (for the purpose of our work, we will use the IDLE debugger).

Note: debugging is not testing, it is a tool used by developers to investigate suspected defects.

- Debugging is done by developers, it is not testing.
- Unit testing is implemented by developers, it is testing!
- Confused? Let's take a closer look.

# Debugging & Defects

---

If you enter the wrong code, the computer won't give you the right program.

Computer programs do what you tell them to, but what you tell the program to do may not be the same as what you *wanted* the program to do.

This gives rise to a particular type of defect – a defect introduced by a programmer wherein the programmer's expectation of their (own) code is not consistent with the execution of the corresponding program.

Debuggers are tools used by developers to investigate this particular type of defect.

It is *sometimes* the quickest way for a developer to examine and resolve the defect.

# Debugging

---

It can be difficult to figure out how your code can be causing a defect.

Lines of code get executed quickly, values in variables change frequently.

The debugger is a program that enables developers to *step through* their code one line at a time in the same order that they are executed.

The debugger also identifies the values stored in variables at each step.

To speed up the debugging process, it is not necessary to step through each and every line – we can jump to specific code of interest to our investigation (which can be very efficient in terms of identifying the root cause of defects).

# Getting set up in IDLE

---

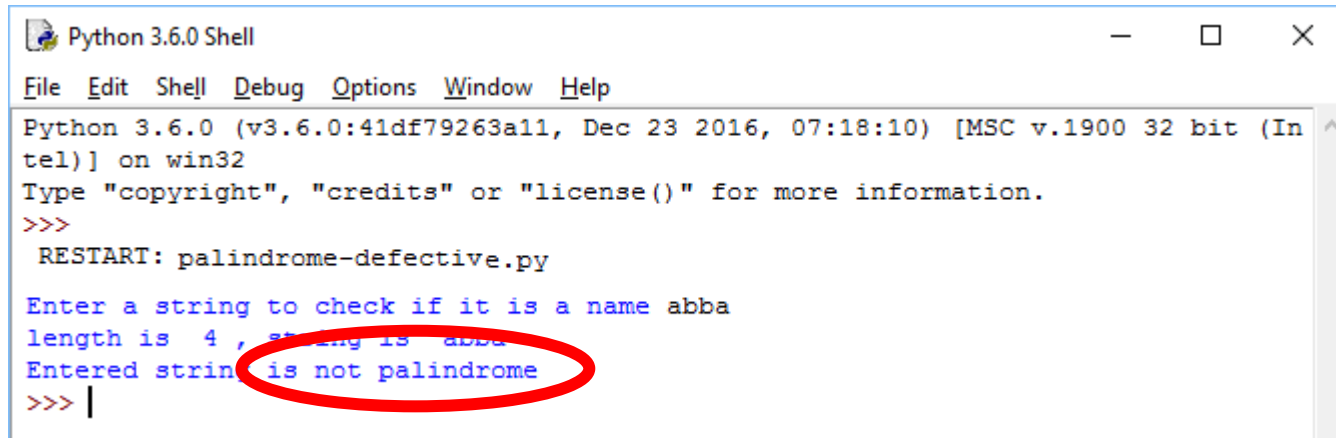
Open the sample defective source code provided in Loop:  
*palindrome-defective.py*

Execute the programme in IDLE (either press F5 or Menu option  
“Run”, sub option “Run Module”)

Test the execution of the programme by entering a valid palindrome,  
e.g. “abba”

Observe that the program does not function as expected, it reports  
that the string “abba” is not a palindrome (which is incorrect).

# Observe that the program does not function as expected



```
Python 3.6.0 Shell
File Edit Shell Debug Options Window Help
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: palindrome-defective.py
Enter a string to check if it is a name abba
length is 4 , string is abba
Entered string is not palindrome
>>> |
```

The screenshot shows a Python 3.6.0 Shell window. The program 'palindrome-defective.py' is running. It prompts the user to 'Enter a string to check if it is a name'. The user enters 'abba'. The program then displays 'length is 4 , string is abba'. The next line, 'Entered string is not palindrome', is circled in red, indicating that the program's output is incorrect for the input 'abba'.

# Our options for investigating this issue in the code?

---

There are essentially three options:

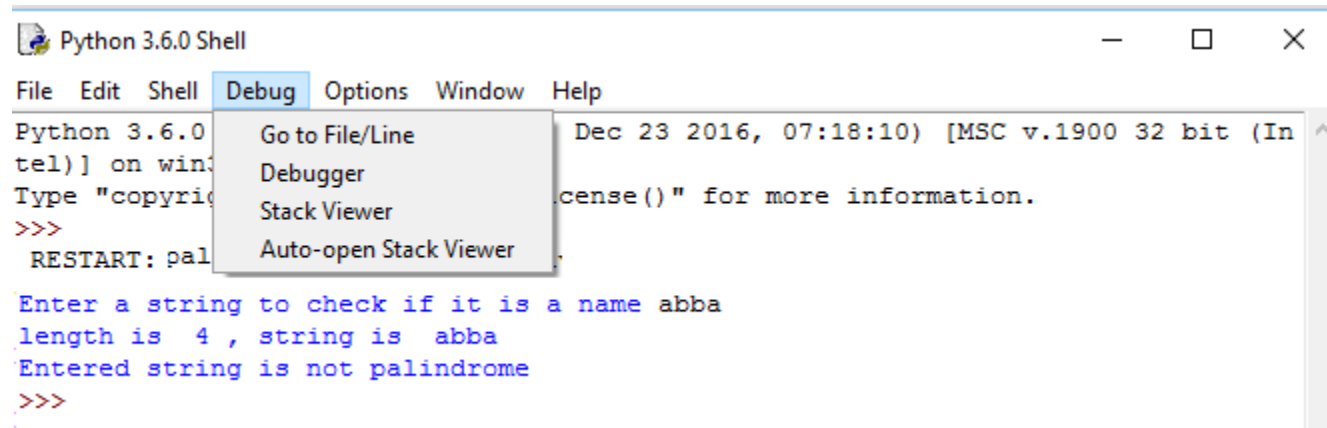
1. Visually review the code
2. Start introducing print statements to try to understand what is happening
3. Use the debugger to try to identify the issue

Limitations of (1) & (2): when there is a lot of code, it can be time consuming getting to review visually.

Benefit of (3): The debugger can *step* straight to the code of interest and *observe the program variables and control flow*.

# Accessing the IDLE Debugger (1)

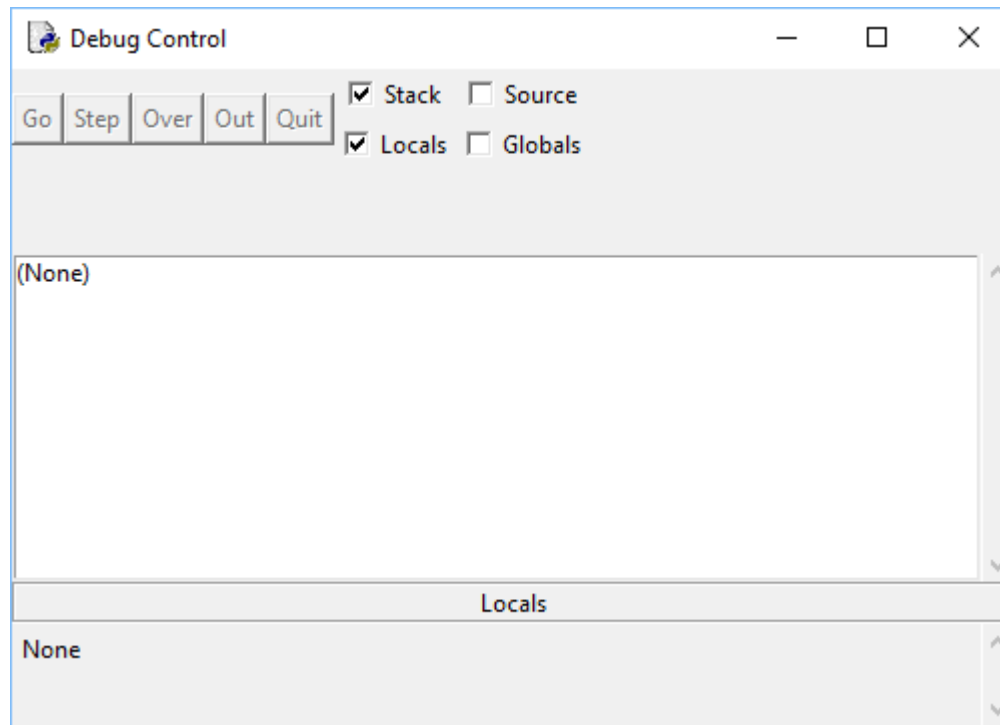
1. In the IDLE Python Shell window (i.e. the window where the code is executed), select menu option “Debug”, sub-option “Debugger”





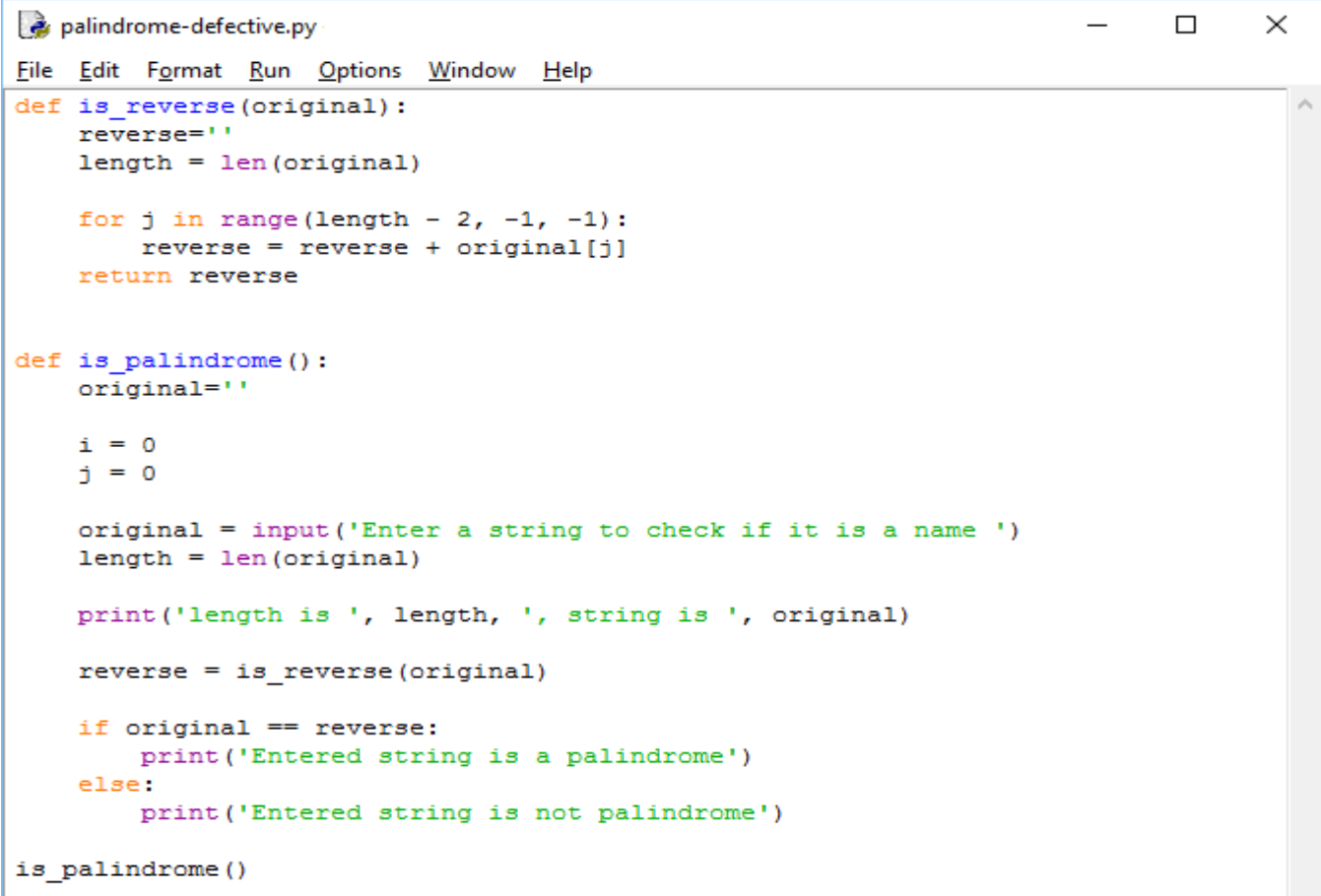
# Accessing the IDLE Debugger (2)

1. The IDLE Debugger is now launched and visible via the Debugger window.



# Debugging the Code (1)

1. Return to the source code window: *palindrome-defective.py* and execute the programme again.



```
palindrome-defective.py
File Edit Format Run Options Window Help
def is_reverse(original):
    reverse=''
    length = len(original)

    for j in range(length - 2, -1, -1):
        reverse = reverse + original[j]
    return reverse

def is_palindrome():
    original=''

    i = 0
    j = 0

    original = input('Enter a string to check if it is a name ')
    length = len(original)

    print('length is ', length, ', string is ', original)

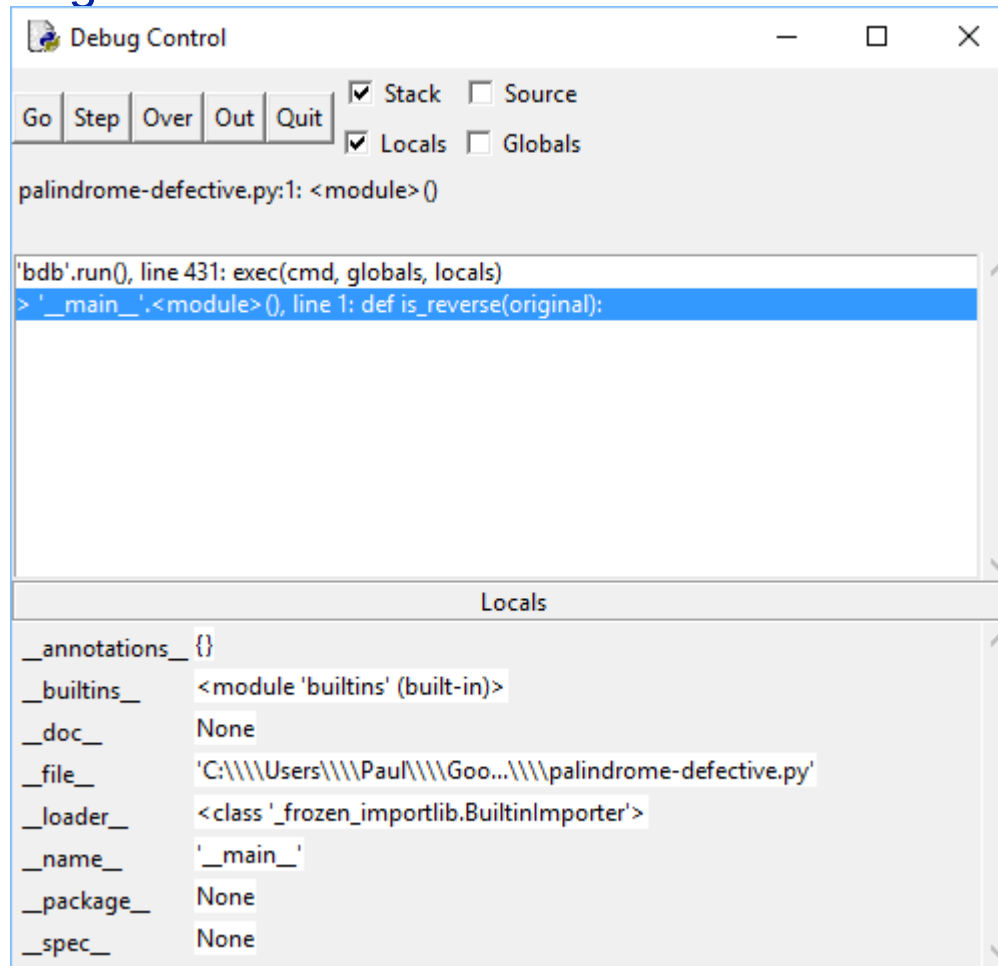
    reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not palindrome')

is_palindrome()
```

# Debugging the Code (2)

2. The debugger activates and automatically stops at the first line of our programme.



# Tip: View the Source Code

2. It is usually helpful to have both the source code and the Debugger in view. (clicking the “source” box does this automatically)

The screenshot displays a Python IDE with two windows. The left window, titled 'palindrome-defective.py', shows the source code of a program. The function `def is_reverse(original):` is circled in red. A red arrow points from this circle to the right window. The right window, titled 'Debug Control', has a 'Source' checkbox circled in red. Below the checkboxes, the current frame is highlighted in blue and circled in red: `> '_main_', <module: 'palindrome-defective.py'> line 1: def is_reverse(original):`. At the bottom of the right window, the 'Locals' pane shows the current state of the program's memory.

```
palindrome-defective.py
File Edit Format Run Options Window Help
def is_reverse(original):
    length = len(original)

    for j in range(length - 2, -1, -1):
        reverse = reverse + original[j]
    return reverse

def is_palindrome():
    original = ''

    i = 0
    j = 0

    original = input('Enter a string to check if it is a palindrome: ')
    length = len(original)

    print('length is ', length, ', string is ', original)

    reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not a palindrome')

is_palindrome()
```

Debug Control

Go Step Over Out Quit ☒ Stack ☐ Source ☒ Locals ☐ Globals

palindrome-defective.py:1: <module>()

'bdb'.run(), line 431: exec(compiled, globals, locals)

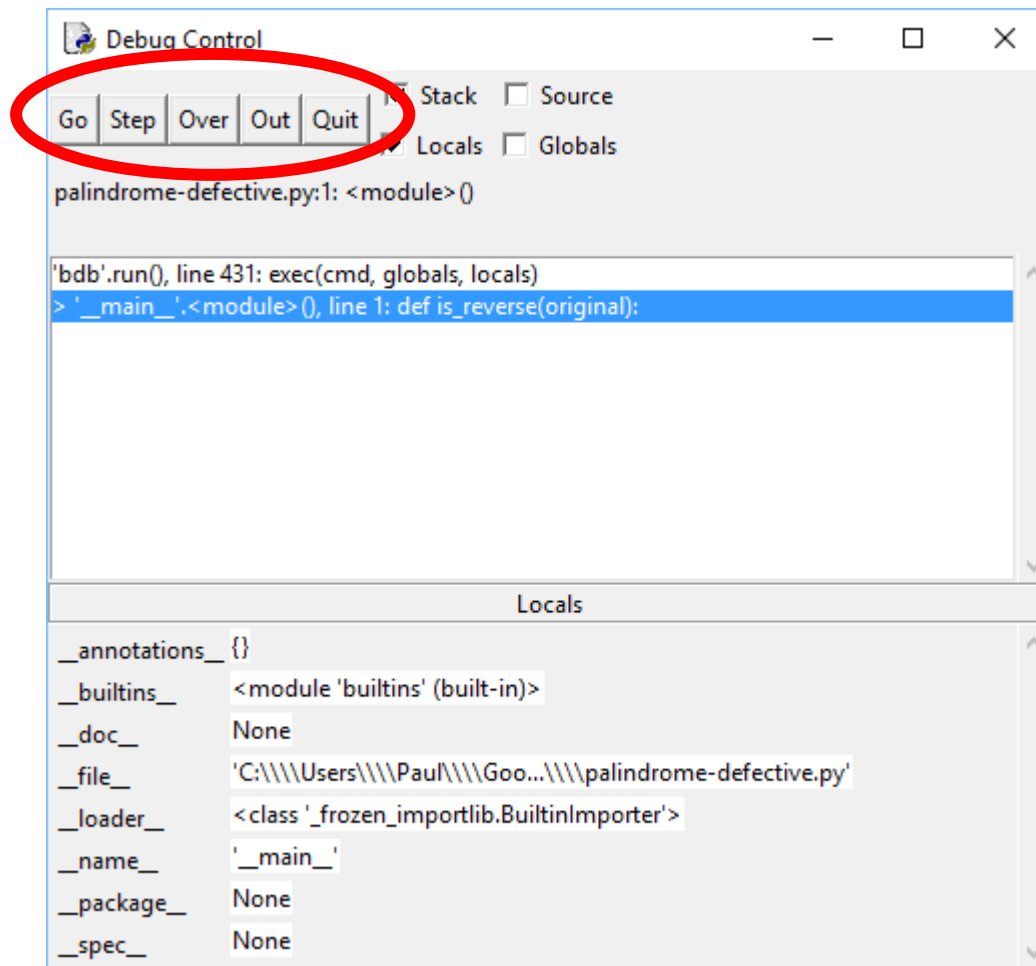
> '\_main\_', <module: 'palindrome-defective.py'> line 1: def is\_reverse(original):

Locals

|                 |                                                    |
|-----------------|----------------------------------------------------|
| __annotations__ | {}                                                 |
| __builtins__    | <module 'builtins' (built-in)>                     |
| __doc__         | None                                               |
| __file__        | 'C:\\Users\\Paul\\Goo...\\palindrome-defective.py' |
| __loader__      | <class 'frozen_importlib.BuiltinImporter'>         |
| __name__        | '_main_'                                           |
| __package__     | None                                               |
| __spec__        | None                                               |

# Debugging the Code (3)

## 3. The 5 IDLE Debugger options.



# Debugging the Code (3)

---

## 3. The 5 IDLE Debugger options.

**Go** - Executes the rest of the code as normal, or until it reaches a *break point* (we discuss break points later.)

**Step** - Step one instruction. If the line contains a function call, the debugger will *step into* the function.

**Over** - Step one instruction. If the line is a function call, the debugger won't *step into* the function, but instead *step over* the call.

**Out** - Keeps stepping over lines of code until the debugger leaves the function it was in when **Out** was clicked. This *steps out* of the function.

**Quit** - Immediately terminates the program.

# Debugging the code (4)

## 4. Stepping over the code

The screenshot displays a Python IDE with a code editor and a Debug Control window. The code editor shows a file named `palindrome-defective.py` with the following code:

```
def is_reverse(original):  
    reverse=''  
    length = len(original)  
  
    for j in range(length - 2, -1, -1):  
        reverse = reverse + original[j]  
    return reverse  
  
def is_palindrome():  
    i = 0  
    j = 0  
  
    original = input('Enter a string to check if it is a palindrome: ')  
    length = len(original)  
  
    print('length is ', length, ', string is ', original)  
  
    reverse = is_reverse(original)  
  
    if original == reverse:  
        print('Entered string is a palindrome')  
    else:  
        print('Entered string is not palindrome')  
  
is_palindrome()
```

The `def is_palindrome():` line is circled in red. The Debug Control window shows the 'Over' button circled in red. The current step is `line 10: def is_palindrome():`, which is also circled in red. A red arrow points from the 'Over' button to the circled function definition in the code editor.

The Debug Control window also shows the following variables in the Locals pane:

| Variable                   | Value                                                                  |
|----------------------------|------------------------------------------------------------------------|
| <code>_annotations_</code> | <code>{}</code>                                                        |
| <code>_builtins_</code>    | <code>&lt;module 'builtins' (built-in)&gt;</code>                      |
| <code>_doc_</code>         | <code>None</code>                                                      |
| <code>_file_</code>        | <code>'C:\\\\Users\\\\Pau\\\\Goo...\\\\palindrome-defective.py'</code> |
| <code>_loader_</code>      | <code>&lt;class 'frozen_importlib.BuiltinImporter'&gt;</code>          |
| <code>_name_</code>        | <code>'_main_'</code>                                                  |
| <code>_package_</code>     | <code>None</code>                                                      |
| <code>_spec_</code>        | <code>None</code>                                                      |
| <code>is_reverse</code>    | <code>&lt;function is_reverse at 0x02F17E88&gt;</code>                 |

# Debugging the code (5)

## 5. Continue stepping over the code

The screenshot shows a Python IDE with a code editor and a Debug Control window. The code editor displays the following code:

```
palindrome-defective.py
File Edit Format Run Options Window Help
def is_reverse(original):
    reverse=''
    length = len(original)

    for j in range(length - 2, -1, -1):
        reverse = reverse + original[j]
    return reverse

def is_palindrome():
    original=''

    i = 0
    j = 0

    original = input('Enter a string to check if it is a palindrome: ')
    length = len(original)

    print('length is ', length, ', string is ', original)

    reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not palindrome')

is_palindrome()
```

The Debug Control window shows the following options:

- Go Stepping Over Out Quit
- Stack Source
- Locals Globals

The current execution state is shown as:

```
palindrome-defective.py:28: <module> ()
'bdb'.run(), line 431: exec(code, globals, locals)
> '_main_.<module> ()', line 28: is_palindrome()
```

The Locals window shows the following variables:

| Variable        | Value                                                     |
|-----------------|-----------------------------------------------------------|
| __annotations__ | {}                                                        |
| __builtins__    | <module 'builtins' (built-in)>                            |
| __doc__         | None                                                      |
| __file__        | 'C:\\\\Users\\\\Pau\\\\Goo...\\\\palindrome-defective.py' |
| __loader__      | <class 'frozen_importlib.BuiltinImporter'>                |
| __name__        | '_main_'                                                  |
| __package__     | None                                                      |
| __spec__        | None                                                      |
| is_palindrome   | <function is_palindrome at 0x033CECD8>                    |
| is_reverse      | <function is_reverse at 0x02F17E88>                       |

A red arrow points from the `is_palindrome()` call in the code editor to the `is_palindrome()` entry in the Locals window.



# Debugging the code (6)

6. Now *step into* the code

The image shows a Python IDE with a code editor and a debug control window. The code editor displays a file named `palindrome-defective.py` with the following code:

```
def is_reverse(original):  
    reverse=''  
    length = len(original)  
  
    for j in range(length - 2, -1, -1):  
        reverse = reverse + original[j]  
    return reverse  
  
def is_palindrome():  
    original=''  
    i = 0  
    j = 0  
  
    original = input('Enter a string to check if it is a palindrome: ')  
    length = len(original)  
  
    print('length is ', length, ', string is ', original)  
  
    reverse = is_reverse(original)  
  
    if original == reverse:  
        print('Entered string is a palindrome')  
    else:  
        print('Entered string is not palindrome')  
  
is_palindrome()
```

The debug control window is titled "Debug Control" and shows the following options:

- ☒ Stack ☐ Source
- ☒ Locals ☐ Globals

The current execution state is shown as:

```
palindrome-defective.py:11: is_palindrome()  
  
'bdb'.run(), line 431: exec(cmd, globals, locals)  
'_main_'.<module>0: line 10: is_palindrome()  
> '_main_'.is_palindrome(), line 11: original=
```

A red circle highlights the "Step" button in the debug control window. A red arrow points from the "Step" button to the line `original=''` in the code editor, indicating the next step in the debugging process.

# Debugging the code (7)

7. And now *step over* the code again...

The screenshot displays a Python IDE with a code editor and a Debug Control window. The code editor shows the following code:

```
def is_reverse(original):  
    reverse=''  
    length = len(original)  
  
    for j in range(length - 2, -1, -1):  
        reverse = reverse + original[j]  
    return reverse  
  
def is_palindrome():  
    original = input('Enter a string to check if it is a palindrome: ')  
    length = len(original)  
  
    print('length is ', length, ', string is ', original)  
  
    reverse = is_reverse(original)  
  
    if original == reverse:  
        print('Entered string is a palindrome')  
    else:  
        print('Entered string is not palindrome')  
  
is_palindrome()
```

The Debug Control window shows the following controls and state:

- Buttons: Go, Step, **Over**, Out, Quit. The 'Over' button is circled in red.
- Checkboxes: ☒ Stack, ☐ Source, ☒ Locals, ☐ Globals.
- Current execution state: `palindrome-defective.py:13: is_palindrome()`
- Stack trace:
  - 'bdb'.run(), line 431: exec(cmd, globals, locals)
  - '\_main\_'.<module>:0: line 20: is\_palindrome()
  - > '\_main\_'.is\_palindrome(), line 13: i = 0
- Locals panel: `original ''`

A red circle highlights the line `i = 0` in the `is_palindrome()` function in the code editor. A red arrow points from the 'Over' button in the Debug Control window to this line.

# Debugging the code (8)

8. And now *step over* the code again...

The screenshot shows a Python IDE with a file named `palindrome-defective.py`. The code defines two functions: `is_reverse` and `is_palindrome`. The `is_palindrome` function is currently being debugged. The `Debug Control` window is open, showing the `Over` button selected. The `Locals` checkbox is checked, and the `Locals` panel displays the current state of local variables: `i` is 0 and `original` is an empty string. A red arrow points from the `j = 0` line in the code to the `Locals` panel.

```
def is_reverse(original):
    reverse=''
    length = len(original)

    for j in range(length - 2, -1, -1):
        reverse = reverse + original[j]
    return reverse

def is_palindrome():
    original=''

    i = 0
    j = 0

    original = input('Enter a string to check if i
length = len(original)

    print('length is ', length, ', string is ' or
reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not palindrome')

is_palindrome()
```

Note: The value of local variables is displayed here so long as the “Locals” box is selected in the Debugger,

# Debugging the Code – breakpoints

## (i)

---

- But stepping *over* (and especially *into*) all of the code is not necessary to get to the problem area – and it is far too inefficient.
- In the case of our code, we know that there is an issue when checking if a string is a palindrome.
- We can therefore often ignore all of the code leading up to the problem area.
- This is achieved through the use of *breakpoints*:
  - First, we set a breakpoint in the part of the code of direct interest
  - Second, we jump directly to that breakpoint by using the “Go” option
  - Recall that “Go” will simply execute the code either to completion or to the first breakpoint it encounters.

# Debugging the Code – breakpoints

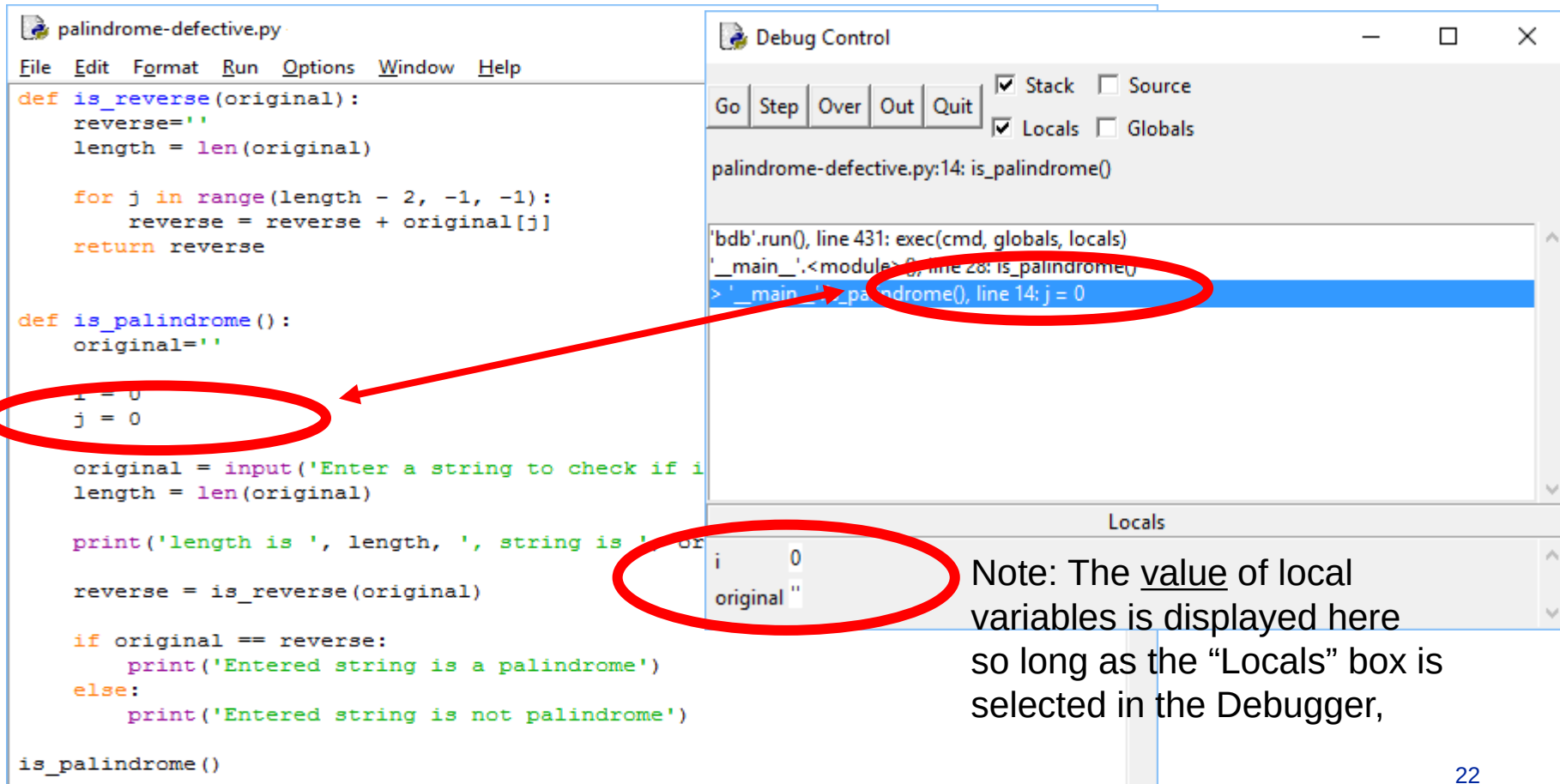
## (ii)

---

- To set a break point, right-click on the line in the file editor and select **Set Breakpoint** from the menu that appears.
- The file editor will highlight that line with yellow.
- You can set break points on as many lines as you want.
- To remove the break point, click on the line and select **Clear Breakpoint** from the menu that appears.
- In practice, programmers set the breakpoint(s) of interest without the need to step into or over any code that they feel will not bring them directly to the problem area... and keep doing this until the source of the defect is identified.

# Debugging the code (9)

## 9. Setting our breakpoint... in the is\_reverse() function



The screenshot displays a Python IDE with a code editor and a Debug Control window. The code editor shows the following code:

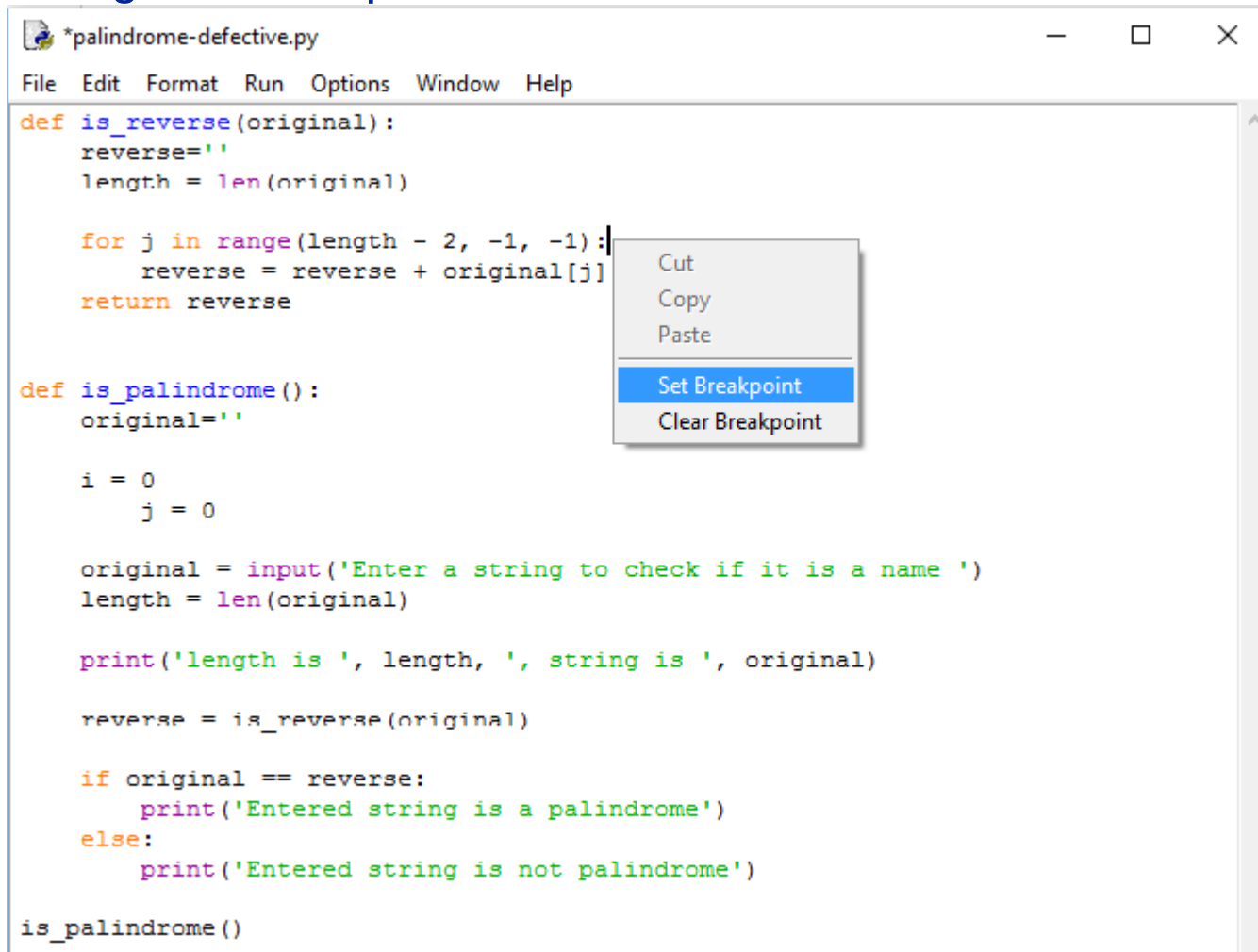
```
def is_reverse(original):  
    reverse=''  
    length = len(original)  
  
    for j in range(length - 2, -1, -1):  
        reverse = reverse + original[j]  
    return reverse  
  
def is_palindrome():  
    original=''  
    i = 0  
    j = 0  
  
    original = input('Enter a string to check if it is a palindrome: ')  
    length = len(original)  
  
    print('length is ', length, ', string is ', original)  
  
    reverse = is_reverse(original)  
  
    if original == reverse:  
        print('Entered string is a palindrome')  
    else:  
        print('Entered string is not palindrome')  
  
is_palindrome()
```

The Debug Control window shows the current execution state. The breakpoint is set at line 14 of the is\_palindrome function, where j = 0. The Locals window shows the current values of local variables i and original.

Note: The value of local variables is displayed here so long as the “Locals” box is selected in the Debugger,

# Debugging the code (10)

## 10. Setting our breakpoint...



```
*palindrome-defective.py
File Edit Format Run Options Window Help

def is_reverse(original):
    reverse=''
    length = len(original)

    for j in range(length - 2, -1, -1):
        reverse = reverse + original[j]
    return reverse

def is_palindrome():
    original=''

    i = 0
    j = 0

    original = input('Enter a string to check if it is a name ')
    length = len(original)

    print('length is ', length, ', string is ', original)

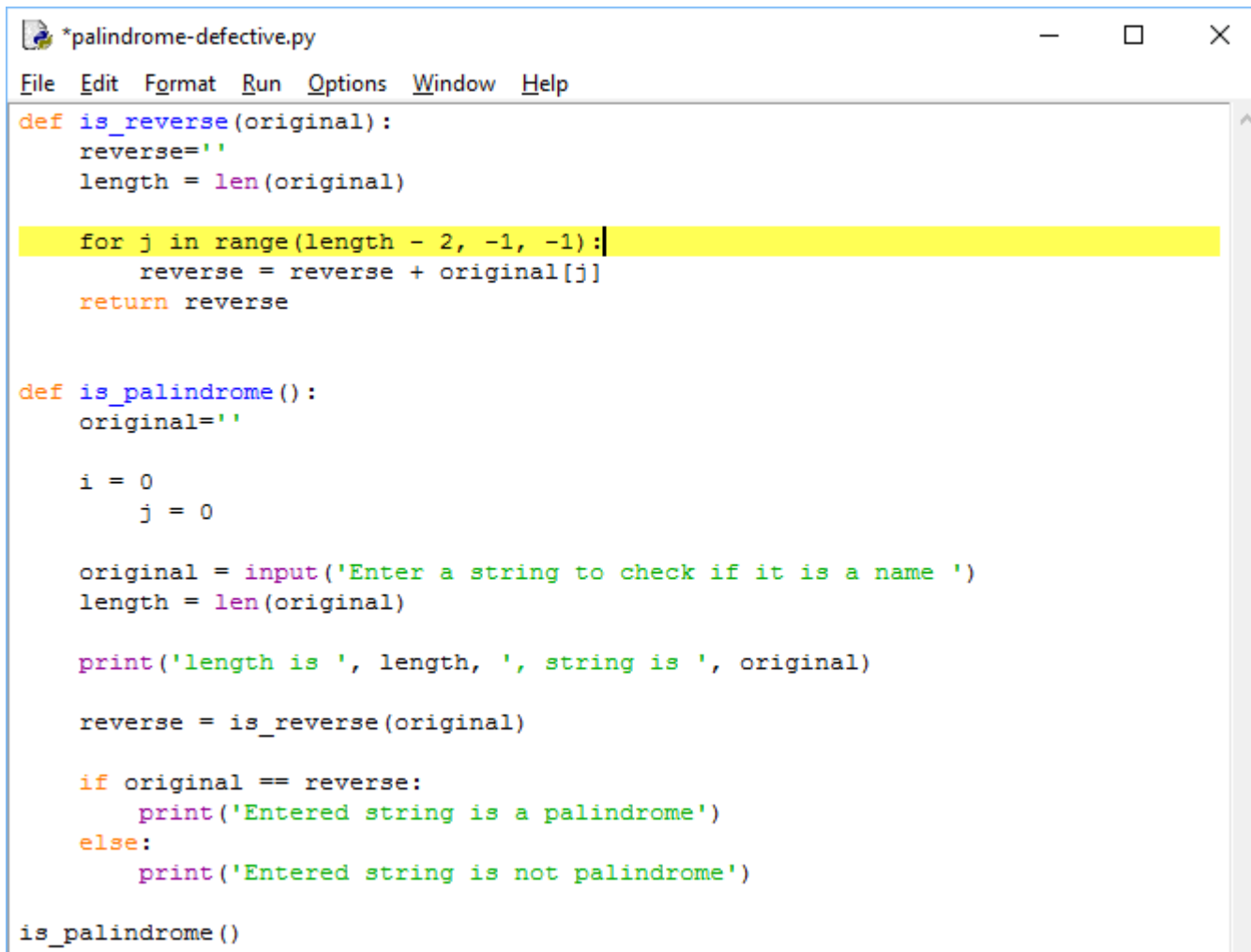
    reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not palindrome')

is_palindrome()
```

# Debugging the code (10)

## 10. Our breakpoint is now set...



```
*palindrome-defective.py
File Edit Format Run Options Window Help
def is_reverse(original):
    reverse=''
    length = len(original)

    for j in range(length - 2, -1, -1):
        reverse = reverse + original[j]
    return reverse

def is_palindrome():
    original=''

    i = 0
    j = 0

    original = input('Enter a string to check if it is a name ')
    length = len(original)

    print('length is ', length, ', string is ', original)

    reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not palindrome')

is_palindrome()
```



# Debugging the code (11)

## 11. "Go" directly to our breakpoint

The screenshot displays a Python IDE with a code editor and a Debug Control window. The code editor shows the following code:

```
def is_reverse(original):  
    reverse=''  
    length = len(original)  
    for j in range(length - 2, -1, -1):  
        reverse = original[j] + reverse  
    return reverse  
  
def is_palindrome():  
    original=''  
  
    i = 0  
    j = 0  
  
    original = input('Enter a string to check if it is a palindrome: ')  
    length = len(original)  
  
    print('length is ', length, ', string is ', original)  
  
    reverse = is_reverse(original)  
  
    if original == reverse:  
        print('Entered string is a palindrome')  
    else:  
        print('Entered string is not palindrome')  
  
is_palindrome()
```

The Debug Control window shows the following options:

- ☒ Stack ☐ Source
- ☒ Locals ☐ Globals

The 'Go' button is circled in red, and an arrow points from it to the breakpoint in the code. The stack trace shows the following frames:

- 'bdb'.run(), line 431: exec(cmd, globals, locals)
- '\_main\_'.<module>(), line 28: is\_palindrome()
- > '\_main\_.is\_palindrome(), line 14: j = 0

The Locals window shows the following variables:

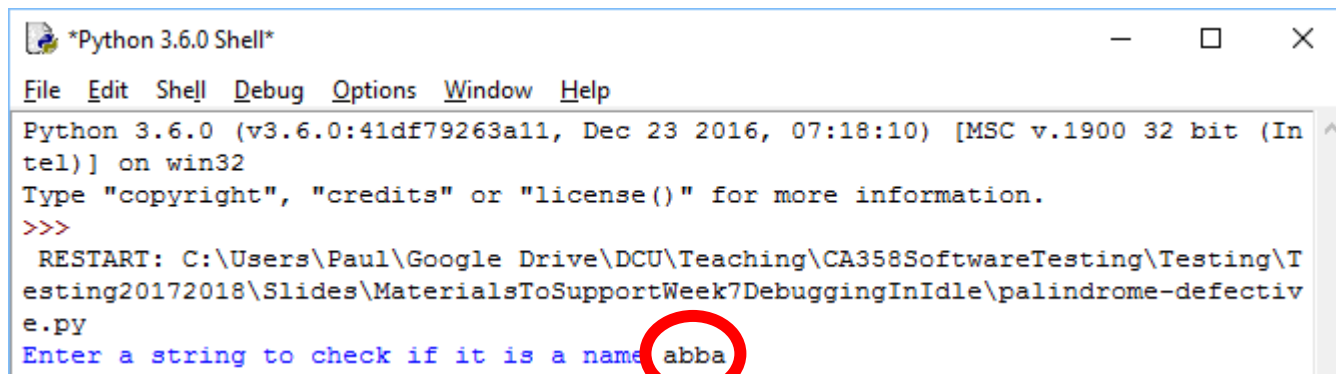
- i: 0
- original: ''

# Debugging the code (12)

12. In the case of our code, going directly to our breakpoint now involves entering a candidate string in the Python Shell.

We enter the string “abba”...

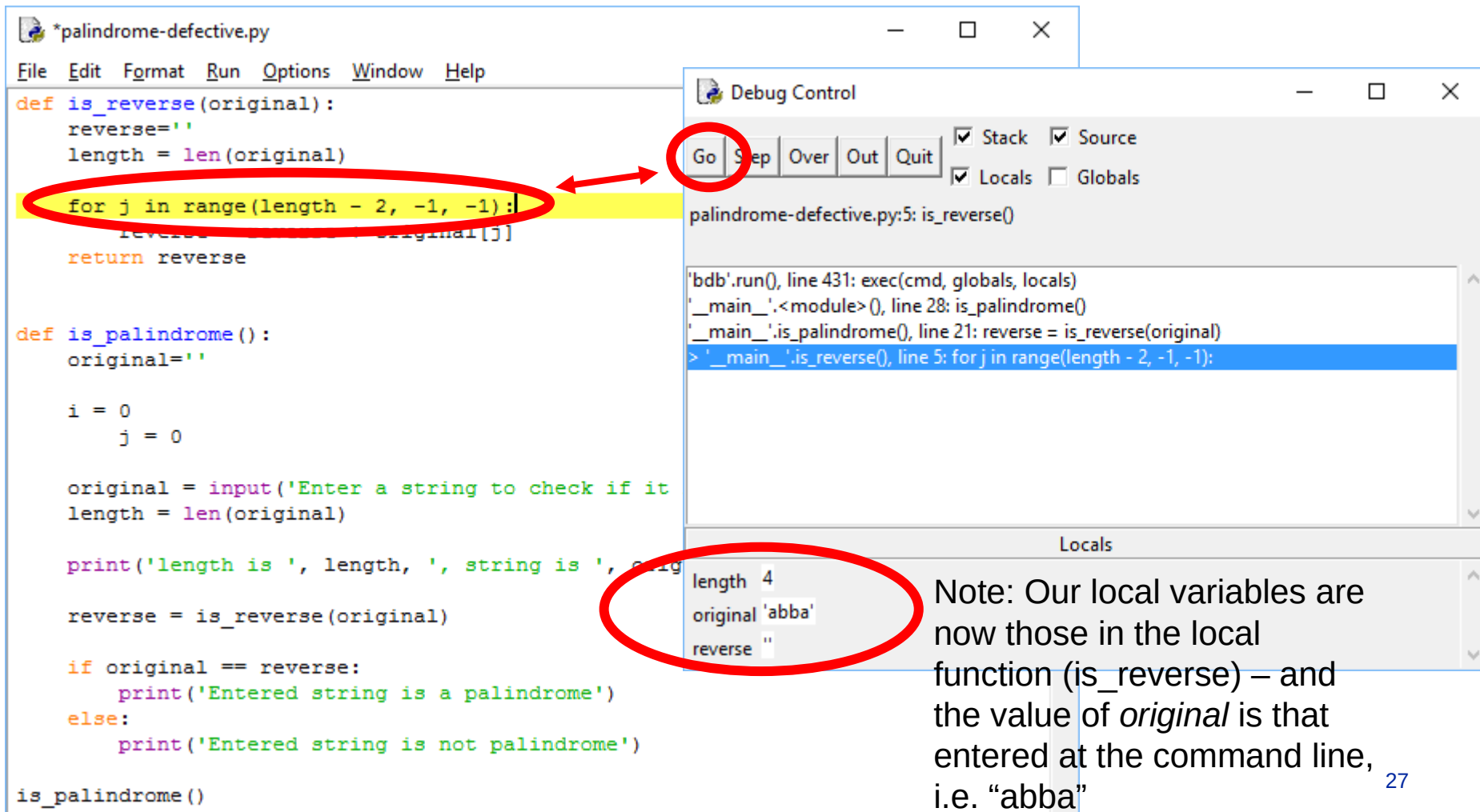
**DON'T FORGET TO ENTER A STRING!**



```
*Python 3.6.0 Shell*
File Edit Shell Debug Options Window Help
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:\Users\Paul\Google Drive\DCU\Teaching\CA358SoftwareTesting\Testing\Testing20172018\Slides\MaterialsToSupportWeek7DebuggingInIdle\palindrome-defective.py
Enter a string to check if it is a name: abba
```

# Debugging the code (13)

## 13. Debugger takes us directly to our breakpoint



The screenshot shows a Python IDE with a file named `*palindrome-defective.py`. The code defines two functions: `is_reverse` and `is_palindrome`. A breakpoint is set on the line `for j in range(length - 2, -1, -1):` in the `is_reverse` function, which is highlighted in yellow. A red circle is drawn around this line, and a red arrow points from it to the `Go` button in the `Debug Control` window. The `Debug Control` window shows the current execution state: `palindrome-defective.py:5: is_reverse()`. The `Locals` window shows the current values of the local variables: `length` is 4, `original` is 'abba', and `reverse` is an empty string. A red circle is drawn around the `Locals` window. A note explains that the local variables are now those in the local function (`is_reverse`) and that the value of `original` is the one entered at the command line, i.e. "abba".

```
def is_reverse(original):
    reverse=''
    length = len(original)
    for j in range(length - 2, -1, -1):
        reverse = original[j] + reverse
    return reverse

def is_palindrome():
    original=''

    i = 0
    j = 0

    original = input('Enter a string to check if it is a palindrome: ')
    length = len(original)

    print('length is ', length, ', string is ', original)

    reverse = is_reverse(original)

    if original == reverse:
        print('Entered string is a palindrome')
    else:
        print('Entered string is not palindrome')

is_palindrome()
```

Debug Control

Go Step Over Out Quit

Stack Source  
Locals Globals

palindrome-defective.py:5: is\_reverse()

'bdb'.run(), line 431: exec(cmd, globals, locals)  
'\_\_main\_\_'.<module>(), line 28: is\_palindrome()  
'\_\_main\_\_'.is\_palindrome(), line 21: reverse = is\_reverse(original)  
> '\_\_main\_\_'.is\_reverse(), line 5: for j in range(length - 2, -1, -1):

Locals

length 4  
original 'abba'  
reverse ''

Note: Our local variables are now those in the local function (`is_reverse`) – and the value of `original` is that entered at the command line, i.e. "abba"

# Debugging the problem

---

- Now that we have got close to where we feel the source of the problem exists, we want to:
  - Step over the lines of code and examine the changes to the local variables
  - In particular in our case, we are interested to see how the string *reverse* is built, as it is a strong potential source for our defect.
  - So, we step over the code....
  - And immediately we notice an issue – the first letter in reverse is “b” and not “a” as was expected.

# Debugging the code (13)

13. Step over the code twice (i.e. click “step” twice thereby bringing us back to the “for” statement)....

```
def is_reverse(original):  
    reverse=''  
    length = len(original)  
    for j in range(length - 2, -1, -1):  
        reverse = reverse + original[j]  
    return reverse  
  
def is_palindrome():  
    original=''  
  
    i = 0  
    j = 0  
  
    original = input('Enter a string to check if it is a palindrome: ')  
    length = len(original)  
  
    print('length is ', length, ', string is ', original)  
  
    reverse = is_reverse(original)  
  
    if original == reverse:  
        print('Entered string is a palindrome')  
    else:  
        print('Entered string is not palindrome')  
  
is_palindrome()
```

Debug Control

Go Step Over Out Quit

Stack Source  
Locals Globals

palindrome-defective.py:6: is\_reverse()

'bdb'.run(), line 431: exec(cmd, globals, locals)  
'\_main\_'.<module>(), line 28: is\_palindrome()  
'\_main\_'.is\_palindrome(), line 21: reverse = is\_reverse(original)  
> '\_main\_'.is\_reverse(), line 6: reverse = reverse + original[j]

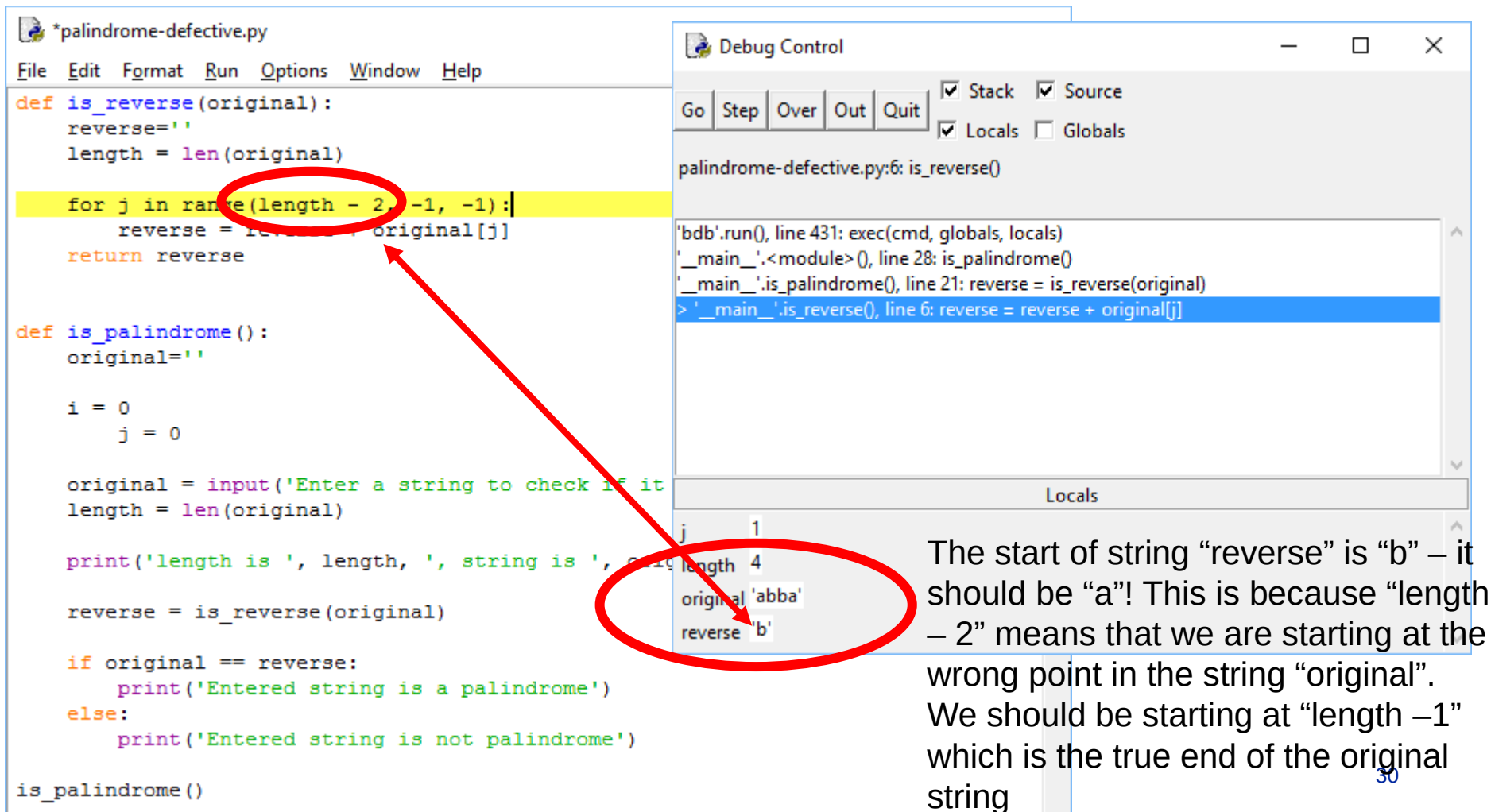
Locals

j 1  
length 4  
original 'abba'  
reverse 'b'

Note: the start of string “reverse” is “b” – it should be “a”! Why is this?

# Debugging the code (14)

## 14. The source of the defect is revealed



The screenshot shows a Python IDE with a file named `*palindrome-defective.py`. The code defines two functions: `is_reverse` and `is_palindrome`. The `is_reverse` function has a loop `for j in range(length - 2, -1, -1):` which is highlighted in yellow. A red circle is drawn around `length - 2` in the code. A red arrow points from this circle to a red circle in the 'Locals' window. The 'Locals' window shows the following variables:

| Variable | Value  |
|----------|--------|
| j        | 1      |
| length   | 4      |
| original | 'abba' |
| reverse  | 'b'    |

The 'Debug Control' window shows the current execution state:

- Go Step Over Out Quit
- ☒ Stack ☒ Source
- ☒ Locals ☐ Globals
- palindrome-defective.py:6: is\_reverse()
- 'bdb'.run(), line 431: exec(cmd, globals, locals)
- '\_main\_'.<module>(), line 28: is\_palindrome()
- '\_main\_'.is\_palindrome(), line 21: reverse = is\_reverse(original)
- > '\_main\_'.is\_reverse(), line 6: reverse = reverse + original[j]

The start of string "reverse" is "b" – it should be "a"! This is because "length - 2" means that we are starting at the wrong point in the string "original". We should be starting at "length - 1" which is the true end of the original string

# Debugging the problem

---

- And we can use this information to fix the defect.
- Change “length – 2” to “length -1”.
- Run the program and see if it now works.... And it does.
- Most programmers would clear the breakpoint – and probably also exit the debugger at this point and simply execute the code as normal, expecting the defect to be fixed.
- This is the power of the debugger!

# Over to you

---

Any Questions?